Step 1 — A Graph With Only Short-Term (Checkpointer) Memory — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds a normal chat graph with only a `SqliteSaver` checkpointer attached — it genuinely remembers everything *within* one conversation thread, but forgets completely the moment you start a new `thread_id`, because a checkpointer's memory is scoped to one thread, not shared across them. **If not:** you'd never actually feel the real gap that the long-term Store in later steps exists to fill.

Setup files

`requirements.txt`: `langgraph`, `langgraph-checkpoint-sqlite`, `langchain-openai`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `*.sqlite`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
```

```
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py

def load_prompt(name):
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The state

```
# src/state.py
# ChatState flows through the graph. messages uses add_messages so each node
# can append. user_id identifies the person (for the Store, Step 2).
# recalled holds the facts the Store returned (filled in Step 2).
from typing import Annotated, TypedDict

from langgraph.graph.message import add_messages

class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
    user_id: str
    recalled: str
```

The chat prompt and the chat node

```
# src/prompts/chat.txt
You are MemoryKeeper, a friendly assistant. Answer naturally and briefly.
What you remember about this user (may be empty):
{recalled}
```

```
# src/nodes.py  (Step 1 version — chat only)
# chat_node: fold whatever is in state["recalled"] into the system prompt,
# then reply. The model is bound in build_graph (a closure), so the node
# takes only state.
from langchain_core.messages import SystemMessage

from src.utils.prompts import load_prompt

def make_chat_node(model):
    def chat_node(state):
        recalled = state.get("recalled") or "nothing yet"
        system = load_prompt("chat").replace("{recalled}", recalled)
        messages = [SystemMessage(content=system)] + state["messages"]
        reply = model.invoke(messages)
        return {"messages": [reply]}
    return chat_node
```

The graph

```
# src/graph.py  (Step 1 version)
# build_graph: START -> chat -> END, compiled with the checkpointer (and the
# store, which is unused until Step 2). The model is bound into the chat node.
from langgraph.graph import END, START, StateGraph

from src.nodes import make_chat_node
from src.state import ChatState

def build_graph(model, store, checkpointer):
    graph = StateGraph(ChatState)
    graph.add_node("chat", make_chat_node(model))
    graph.add_edge(START, "chat")
    graph.add_edge("chat", END)
    return graph.compile(checkpointer=checkpointer, store=store)
```

main.py

```
# main.py  (Step 1 version)
# Build the model and a file-backed SqliteSaver (inside its `with` block, so
# a conversation survives a restart), then a terminal loop on a hardcoded
# thread_id. Ask your name, then ask it back -> it remembers. Start a new
# process with a new thread_id -> it forgets (the gap Step 2 fixes).
import sys

from langchain_core.messages import HumanMessage
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.store.memory import InMemoryStore

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.models.llm import get_model

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    model = get_model(config)
    store = InMemoryStore()
    with SqliteSaver.from_conn_string("memorykeeper.sqlite") as checkpointer:
        app = build_graph(model, store, checkpointer)
        print("MemoryKeeper — type a message, or /quit to exit.")
```

```
        run_config = {"configurable": {"thread_id": "demo-thread"}}
        while True:
            text = input("You: ").strip()
            if text == "/quit":
                break
            if text == "":
                continue
            result = app.invoke(
                {"messages": [HumanMessage(content=text)],
                 "user_id": "sam", "recalled": ""},
                run_config,
            )
            print("MemoryKeeper:", result["messages"][-1].content)

if __name__ == "__main__":
    main()
```

The test

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MEMORYKEEPER_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")
```

```
def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

Run `main.py`, tell it your name, ask it back in the same thread — it remembers (the checkpointer). Stop and restart with a new `THREAD_ID` — it's a stranger again. That amnesia is exactly what Step 2's Store fixes.